

Билет 15. Файберы. Идея реализации context switch. Scheduler, `yield`, fault injection

0. Формулировка билета

Файберы. Идея реализации context switch. Scheduler, функция `yield`. Fault injection.

Главная идея:

Файбер — это легковесный поток исполнения в user space.
Он похож на обычный поток тем, что у него есть свой стек и точка исполнения, но переключает его не ОС, а наш runtime.

Обычный поток ОС:

```
планируется ядром  
имеет стек  
может быть вытеснен таймером  
context switch делает ОС
```

Файбер:

```
планируется библиотекой в user space  
имеет свой стек  
переключается добровольно  
context switch делает наш код
```

Коротко:

```
thread = kernel/user-space execution unit, планируется ОС  
fiber  = user-space execution unit, планируется нашим scheduler-ом
```

1. Зачем нужны файберы

До файберов у нас были три подхода.

1. Поток на клиента

```
while (true) {  
    int client_fd = accept(server_fd, nullptr, nullptr);
```

```
std::thread(handle_client, client_fd).detach();  
}
```

Плюс:

код простой
каждый клиент пишется как обычная последовательная функция

Например:

```
void session(int fd) {  
    while (true) {  
        auto request = read_request(fd);  
        auto response = process(request);  
        write_response(fd, response);  
    }  
}
```

Минус:

поток ОС дорогой
у каждого потока большой стек
много context switch через ядро
C10k плохо масштабируется

2. `epoll` + `callbacks`

```
async_read(fd, [](Result r) {  
    async_write(fd, response, [](Result r) {  
        async_read(fd, ...);  
    });  
});
```

Плюс:

один поток может обслуживать много соединений
нет потока ОС на каждого клиента

Минус:

callback hell
логика разорвана на куски

локальные переменные приходится переносить в объект `session`
нужно вручную следить за `lifetime` через `shared_ptr`

3. Futures

```
ReadAsync(fd)
    .ThenValue(Process)
    .ThenValue(WriteAsync);
```

Плюс:

лучше композиция
можно строить цепочки и комбинаторы

Минус:

это `data flow` стиль
не всегда удобно писать обычную бизнес-логику
всё равно не выглядит как простой синхронный код

Файберы

Файберы хотят дать лучшее из двух миров:

код выглядит как синхронный
исполняется как асинхронный
поток ОС не блокируется

Пользователь хочет писать:

```
void session(Socket s) {
    while (true) {
        auto request = Read(s);
        auto response = Process(request);
        Write(s, response);
    }
}
```

Но под капотом:

Read не блокирует поток ОС
Read паркует текущий файбер
scheduler запускает другой файбер
когда fd готов, scheduler возобновляет первый файбер

2. Что такое файбер

Файбер можно понимать как “поток без ядра”.

У файбера есть:

свой стек
сохранённый stack pointer
сохранённые регистры
функция, которую он выполняет
состояние: runnable / running / suspended / dead

Но у него нет:

собственного kernel thread
вытеснения таймером ОС
системного планирования

Файберы обычно работают поверх ограниченного числа потоков ОС.

Например:

4 kernel threads
100000 fibers

Каждый поток ОС по очереди исполняет разные файберы.

3. Thread vs Fiber

Свойство	Thread	Fiber
Планировщик	ОС	user-space runtime
Переключение	через ядро	в user space
Стек	есть	есть
Вытеснение	preemptive	обычно cooperative

Свойство	Thread	Fiber
Блокирующий syscall	блокирует поток ОС	нельзя напрямую блокировать
Масштаб	тысячи тяжело	сотни тысяч возможно
Код	синхронный	синхронный стиль

Главное отличие:

```
thread блокируется → ядро паркует поток
fiber блокируется → runtime паркует файбер, но поток ОС продолжает работать
```

4. Cooperative scheduling

Файберы обычно кооперативные.

Это значит:

Файбер сам должен добровольно отдать управление.

Например:

```
yield();
```

или:

```
Read(fd); // внутри может suspend
```

или:

```
MutexLock(); // если mutex занят, suspend
```

В отличие от thread:

```
thread может быть вытеснен таймером ОС в любой момент
```

fiber:

```
переключается только в известных точках
```

Плюсы:

меньше неожиданной недетерминированности
дешёвое переключение
проще reasoning внутри маленьких участков без yield

Минусы:

если файбер долго считает и не вызывает yield,
он заблокирует весь scheduler thread

Поэтому файберы хороши для:

IO-bound задач
множества соединений
ожидания событий
кооперативной синхронизации

И хуже подходят для:

долгих CPU-bound циклов без yield

5. Что такое context

Чтобы приостановить функцию и потом продолжить с того же места, нужно сохранить её контекст.

Контекст исполнения — это минимальное состояние процессора, достаточное для продолжения.

Обычно нужно сохранить:

stack pointer
callee-saved registers
instruction pointer косвенно через ret address
иногда sanitizer state

Почему важен stack pointer?

Потому что стек содержит:

локальные переменные
адреса возврата

сохранённые регистры
цепочку вызовов

Если у каждого фибера свой стек, то переключение фибера — это в первую очередь:

сохранить текущий RSP
загрузить RSP другого фибера
восстановить регистры
ret

После `ret` процессор продолжит выполнение уже на другом стеке.

6. Почему фиберу нужен свой стек

Допустим, фибер выполняет:

```
void session(Socket s) {  
    std::string request = Read(s);  
    std::string response = Process(request);  
    Write(s, response);  
}
```

Если внутри `Read(s)` фибер приостановился, нужно сохранить:

что мы внутри `session`
что мы внутри `Read`
локальную переменную `request` ещё нет или она частично строится
адрес возврата из `Read` в `session`
адрес возврата из `session` в `trampoline`

Всё это естественно лежит на стеке.

Поэтому `stackful`-фибер сохраняет весь стек вызовов.

Это важное отличие от `stackless`-корутин:

`fiber / stackful coroutine`:
сохраняет весь стек вызовов

`C++20 coroutine / stackless`:
компилятор выносит состояние одной `coroutine`-функции во `frame`

7. ExecutionContext

Можно завести класс:

```
class ExecutionContext {
public:
    void Setup(void* stack_bottom, void (*trampoline)(void*), void* arg);
    void SwitchTo(ExecutionContext& target);

private:
    void* rsp_ = nullptr;
};
```

Смысл:

ExecutionContext хранит сохранённый stack pointer
Setup подготавливает новый стек
SwitchTo переключает текущий контекст на другой

Файбер содержит:

```
class Fiber {
private:
    ExecutionContext context_;
    Stack stack_;
    Function routine_;
};
```

Scheduler тоже имеет свой контекст:

scheduler context
fiber context #1
fiber context #2
fiber context #3

Когда файбер вызывает `yield`, происходит:

fiber context → scheduler context

Когда scheduler выбирает файбер:

scheduler context → fiber context

8. Идея SwitchContext

На x86-64 System V callee-saved registers:

```
rbx
rbp
r12
r13
r14
r15
rsp
```

Идея ассемблерной функции:

```
extern "C" void SwitchContext(void** from_rsp, void* to_rsp);
```

Смысл аргументов:

```
from_rsp – куда сохранить текущий RSP
to_rsp   – какой RSP загрузить
```

Упрощённая схема:

```
SwitchContext:
    push r15
    push r14
    push r13
    push r12
    push rbx
    push rbp

    mov [rdi], rsp    ; *from_rsp = rsp
    mov rsp, rsi      ; rsp = to_rsp

    pop rbp
    pop rbx
    pop r12
    pop r13
    pop r14
    pop r15

    ret
```

Что происходит?

1. Сохраняем callee-saved регистры текущего контекста на текущий стек.
2. Сохраняем текущий rsp в from_rsp.
3. Загружаем rsp другого контекста.
4. Восстанавливаем регистры уже с другого стека.
5. ret берёт адрес возврата с другого стека.
6. Исполнение продолжается в другом файбере.

Ключевая мысль:

`ret` после переключения стека возвращает не туда, откуда мы пришли, а туда, где другой контекст был остановлен.

9. Почему instruction pointer не сохраняется явно

Может показаться, что нужно сохранить `RIP`.

Но в такой реализации `RIP` сохраняется неявно.

Когда обычная функция вызывает:

```
SwitchContext(&current.rsp, target.rsp);
```

на стек кладётся адрес возврата.

Когда мы сохраняем `rsp`, этот адрес возврата остаётся на стеке.

Потом, когда контекст возобновят:

```
ret
```

снимет этот старый адрес возврата и продолжит выполнение сразу после того самого вызова `SwitchContext`.

То есть:

```
RIP хранится как return address на стеке
```

10. `SetupContext` : как запустить новый файбер первый раз

С уже запускавшимся файбером понятно:

у него есть сохранённый RSP
на его стеке лежит return address
ret вернёт его туда, где он остановился

Но новый фибер ещё никогда не останавливался.

У него нет сохранённого return address.

Поэтому нужно вручную подготовить стек так, будто он уже был остановлен.

`SetupContext` кладёт на новый стек:

адрес trampoline
начальные значения callee-saved регистров
аргумент для trampoline

Идея:

первый `SwitchContext` на новый фибер
загружает его RSP
восстанавливает fake-регистры
ret прыгает в trampoline

`trampoline` вызывает пользовательскую функцию:

```
void FiberTrampoline(void* arg) {  
    Fiber* fiber = static_cast<Fiber*>(arg);  
  
    fiber->routine_();  
  
    fiber->state_ = Dead;  
  
    Scheduler::Current()->ExitCurrentFiber();  
}
```

11. Почему нужен trampoline

Нельзя просто сделать так, чтобы фибер “вернулся” из пользовательской функции обычным `return`.

Почему?

Пользовательская функция лежит на дне стека фибера.

Если она завершится и выполнит обычный `ret`, она попытается снять со стека адрес возврата.

Но выше неё может не быть корректного адреса.

Поэтому нужен trampoline:

```
trampoline вызывает routine()  
когда routine() завершилась:  
    помечает файбер dead  
    переключается обратно в scheduler
```

То есть файбер при завершении не “возвращается неизвестно куда”.

Он контролируемо сообщает scheduler-у:

```
я закончил работу  
можно меня удалить
```

12. Базовые состояния файбера

Файбер может быть в состояниях:

```
Runnable — готов к запуску  
Running  — сейчас выполняется  
Suspended — ждёт события  
Dead     — завершился
```

Переходы:

```
создали fiber:  
    Runnable  
  
scheduler выбрал:  
    Runnable → Running  
  
fiber yield:  
    Running → Runnable  
  
fiber ждёт I/O/mutex:  
    Running → Suspended  
  
событие произошло:  
    Suspended → Runnable
```

```
routine завершилась:  
Running → Dead
```

13. Scheduler

Scheduler — это объект, который управляет фиберами.

Минимально у него есть:

```
очередь Runnable фиберов  
текущий Running файбер  
scheduler context  
метод Spawn/Go  
метод Run  
метод Yield
```

Пример:

```
class Scheduler {  
public:  
    void Go(Function f);  
    void Run();  
    void Yield();  
  
private:  
    std::deque<Fiber*> run_queue_;  
    Fiber* current_ = nullptr;  
    ExecutionContext context_;  
};
```

Go создаёт новый файбер:

```
создать Fiber  
положить в run_queue
```

Run :

```
пока run_queue не пустая:  
    взять следующий fiber  
    current = fiber  
    switch scheduler → fiber  
    когда fiber вернулся:  
        если Dead — удалить
```

если Runnable — положить обратно
если Suspended — не класть

14. Как работает `yield`

`yield` — добровольная уступка управления.

Пользователь пишет:

```
void f() {  
    std::cout << "A\n";  
    yield();  
    std::cout << "B\n";  
}
```

Смысл:

текущий файбер говорит:
я пока готов продолжать,
но дай поработать другим

Алгоритм:

1. current fiber: Running → Runnable
2. положить его в конец run_queue
3. переключиться fiber context → scheduler context
4. scheduler выберет следующий Runnable fiber

Псевдокод:

```
void Scheduler::Yield() {  
    Fiber* self = current_;  
  
    self->state = State::Runnable;  
    run_queue_.push_back(self);  
  
    SwitchContext(&self->context.rsp, context_.rsp);  
}
```

После того как scheduler когда-нибудь снова выберет этот файбер, `SwitchContext` вернётся, и код продолжится после `yield()`.

15. Пример работы `yield`

Код:

```
go([] {
    std::cout << "A1\n";
    yield();
    std::cout << "A2\n";
});

go([] {
    std::cout << "B1\n";
    yield();
    std::cout << "B2\n";
});

scheduler.Run();
```

Возможный вывод при FIFO scheduler:

```
A1
B1
A2
B2
```

Что произошло:

```
run_queue = [A, B]

scheduler запускает A
A печатает A1
A yield → A уходит в конец очереди

run_queue = [B, A]

scheduler запускает B
B печатает B1
B yield → B уходит в конец

run_queue = [A, B]

scheduler запускает A
A печатает A2
A завершается

scheduler запускает B
```

B печатает B2
B завершается

16. Suspend vs Yield

`yield`:

файбер всё ещё готов продолжать
его можно сразу положить обратно в `run_queue`

`suspend`:

файбер не готов продолжать
он ждёт внешнего события
его нельзя класть в `run_queue`

Пример `yield`:

```
while (has_more_work()) {  
    do_small_step();  
    yield();  
}
```

Пример `suspend`:

```
auto data = Read(fd);
```

Если данных нет:

зарегистрировать `fd` в `epoll`
перевести `fiber` в `Suspended`
переключиться в `scheduler`

Когда `epoll` скажет, что `fd` готов:

`fiber` `Suspended` → `Runnable`
положить в `run_queue`

17. Файберы и `epoll`

Файберы сами по себе дают только:

```
стек
context switch
yield/suspend/resume
```

Но для асинхронного сервера нужен runtime, который знает про I/O.

Например:

```
std::string Read(Socket s) {
    while (true) {
        ssize_t n = read(s.fd, buffer, size);

        if (n > 0) {
            return data;
        }

        if (errno == EAGAIN) {
            scheduler.WaitRead(s.fd);
            continue;
        }

        if (n == 0) {
            throw EOF;
        }

        throw Error;
    }
}
```

`WaitRead(fd)`:

1. зарегистрировать fd в epoll на EPOLLIN
2. связать fd → current fiber
3. current fiber Running → Suspended
4. switch current fiber → scheduler

Scheduler:

```
если run_queue не пустая:
    запустить следующий runnable fiber

если run_queue пустая:
```

```
epoll_wait  
по событиям epoll разбудить suspended fibers
```

Так пользовательский код выглядит блокирующим:

```
auto request = Read(fd);
```

Но поток ОС не блокируется.

Блокируется только файбер.

18. Почему нельзя делать обычный blocking read в файбере

Если внутри файбера вызвать обычный blocking `read(fd)` на blocking socket, то заблокируется не файбер.

Заблокируется весь kernel thread, на котором сейчас выполнялся scheduler.

Это плохо:

```
thread pool потерял один worker thread  
другие файберы на этом потоке не исполняются  
если все scheduler threads так заблокировались – runtime встал
```

Поэтому файберный runtime должен использовать:

```
non-blocking fd  
epoll/io_uring  
fiber-aware mutex/condvar  
fiber-aware sleep/timer
```

Иначе файберы теряют смысл.

19. Fiber mutex

Обычный `std::mutex` при ожидании блокирует поток ОС.

Для файберов это плохо.

Нужен fiber-aware mutex.

Схема:

```
class FiberMutex {
public:
    void Lock();
    void Unlock();

private:
    bool locked_ = false;
    std::queue<Fiber*> waiters_;
};
```

Lock:

```
если mutex свободен:
    locked = true
    продолжить

если mutex занят:
    добавить current fiber в waiters
    current fiber Running → Suspended
    switch в scheduler
```

Unlock:

```
если waiters пуст:
    locked = false

если waiters не пуст:
    взять одного waiter
    перевести его Suspended → Runnable
    положить в run_queue
```

Смысл:

```
ждёт фибер
а не поток ОС
```

Это похоже на futex, но на уровне user-space runtime.

20. Fiber sleep / timer

Обычный:

```
std::this_thread::sleep_for(1s);
```

заблокирует поток ОС.

Файберный sleep должен делать иначе:

```
положить current fiber в таймерную структуру  
Suspended  
switch в scheduler
```

Scheduler при отсутствии runnable fibers ждёт:

```
epoll_wait с timeout до ближайшего timer
```

Когда время пришло:

```
fiber Suspended → Runnable
```

21. Fault injection

Fault injection — это искусственное внесение сбоев или переключений, чтобы проверить корректность concurrent-кода.

В контексте файберов обычно полезна идея:

Мы можем детерминированно вставлять `yield` в разные точки программы и проверять, не ломаются ли инварианты.

Почему это удобно именно с файберами?

Потому что scheduler находится под нашим контролем.

В потоках ОС планировщик недетерминирован:

```
сегодня баг проявился  
завтра нет
```

В файберах можно сделать:

```
после каждой интересной операции попробовать yield  
перебрать разные расписания  
повторить конкретное расписание по seed
```

Это помогает ловить:

```
race condition
deadlock
livelock
нарушения инвариантов
ошибки в lock-free/async коде
ошибки lifetime
```

22. Пример fault injection

Допустим, есть код:

```
void Increment() {
    int tmp = counter;
    counter = tmp + 1;
}
```

Однопоточно всё нормально.

Но если два файбера делают:

```
go(Increment);
go(Increment);
```

и scheduler вставляет `yield` между чтением и записью:

```
fiber A:
    tmp = counter; // tmp = 0
    yield

fiber B:
    tmp = counter; // tmp = 0
    counter = 1
    yield

fiber A:
    counter = 1
```

Итог:

```
counter = 1
```

Хотя ожидали:

```
counter = 2
```

Так мы поймали race condition.

23. Fault injection через random yield

Простейшая идея:

```
void MaybeYield() {  
    if (random()) {  
        yield();  
    }  
}
```

И вставлять:

```
MaybeYield();
```

в критические места:

```
перед/после lock  
перед/после unlock  
перед/после CAS  
перед/после push/pop  
после проверки условия  
перед изменением shared state
```

Плюсы:

```
дешево  
много разных interleavings  
можно воспроизвести по seed
```

Минусы:

```
не полный перебор  
можно не попасть в редкий сценарий
```

24. Deterministic scheduler

Более сильный вариант — управляемый scheduler.

Он может:

```
знать все runnable fibers  
выбирать следующий fiber по seed  
логировать расписание  
потом воспроизводить то же расписание
```

Например:

```
seed = 123  
schedule:  
  A, A, B, A, C, B, ...
```

Если тест упал, можно повторить:

```
run with seed 123
```

Это очень полезно:

```
concurrency bug становится воспроизводимым
```

25. Почему fault injection не равен data race detector

Fault injection и ThreadSanitizer решают разные задачи.

TSan:

```
ловит data race на уровне обращений к памяти
```

Fault injection:

```
проверяет поведение программы на разных расписаниях  
может найти race condition без data race  
может найти deadlock/livelock
```

Пример race condition без data race:

все обращения защищены mutex
но логика check-then-act неправильная

TSan может молчать, но fault injection поймает неправильный результат.

26. Файберы и санитайзеры

Ручное переключение стеков может ломать инструменты.

Почему?

ASan и TSan хранят служебное состояние:

ASan отслеживает границы стеков и red zones
TSan отслеживает happens-before и историю потоков

Если мы просто сменили `RSP`, sanitizer может подумать:

стек внезапно исчез
или обращения идут в неправильном потоке исполнения

Поэтому production-реализации файберов должны уведомлять sanitizer о переключении.

Идея:

перед switch:
 sanitizer_start_switch_fiber

после switch:
 sanitizer_finish_switch_fiber

Для TSan аналогично:

tsan_switch_to_fiber

На экзамене достаточно сказать:

Ручной context switch меняет стек не через ОС, поэтому санитайзерам нужно явно сообщать о переключении fiber context.

27. Многопоточный scheduler

Пока мы говорили про один scheduler thread.

Но можно сделать runtime:

N потоков ОС
общая или локальные очереди runnable fibers
много fibers

Каждый поток ОС:

берёт runnable fiber
resume
fiber yield/suspend/dead
берёт следующий

Архитектуры:

одна глобальная очередь
локальные очереди на worker
work stealing

Глобальная очередь проще, но:

много contention на mutex очереди
хуже масштабирование

Локальные очереди + work stealing:

обычно worker берёт из своей очереди
если пусто – крадёт из чужой

Но это уже следующий уровень: scheduler/multiprocessing.

28. Важная связь: фибер = корутина + runtime

Можно смотреть так:

```
context switch даёт механизм suspend/resume
scheduler решает, кого resume
epoll/timer/mutex решают, когда suspended fiber снова runnable
```

То есть фибер сам по себе — не весь runtime.

Он только даёт возможность:

```
остановить выполнение
сохранить стек
потом продолжить
```

А runtime добавляет:

```
очереди
I/O integration
timers
fiber synchronization
fault injection
multiprocessing
```

29. Типичные ошибки

Ошибка 1. Заблокировать поток ОС

Плохо:

```
std::this_thread::sleep_for(1s);
read(blocking_fd, ...)
std::mutex::lock()
```

внутри фиберного runtime.

Почему плохо?

```
блокируется поток ОС, а не только фибер
```

Ошибка 2. Не сохранить нужные регистры

Если `SwitchContext` не сохраняет callee-saved registers, после возобновления функция может увидеть испорченные значения.

Ошибка 3. Неправильно подготовить начальный стек

Если `SetupContext` неверно положил return address или fake-регистры, первый запуск файбера прыгнет в мусор.

Ошибка 4. Дать пользовательской функции вернуться обычным `ret`

Нужен trampoline, который после завершения routine передаст управление scheduler-у.

Ошибка 5. Удалить файбер, пока он ещё может быть resume

Нужно аккуратно управлять lifetime:

Runnable/Suspended fibers должны жить
Dead fibers можно удалить

Ошибка 6. Не уведомить sanitizer

В учебном коде может быть неважно, но в реальном — ASan/TSan могут начать ругаться неправильно.

30. Что сказать коротко

Файбер — это легковесный user-space поток исполнения. У него есть свой стек и сохранённый контекст, но он не планируется ОС. Переключение файберов делается в user space: мы сохраняем `RSP` и callee-saved регистры текущего файбера, загружаем `RSP` другого файбера, восстанавливаем его регистры и через `ret` продолжаем выполнение с места, где тот был остановлен.

Новый файбер запускается через `SetupContext`: мы вручную готовим его стек так, чтобы первый `SwitchContext` попал в trampoline. Trampoline вызывает пользовательскую функцию, а после её завершения помечает файбер dead и возвращает управление scheduler-у.

Scheduler хранит очередь runnable файберов. `yield` — добровольная уступка: текущий файбер переводится из Running в Runnable, кладётся в конец очереди, и управление переключается в scheduler. Если файбер ждёт I/O, mutex или timer, он переводится в Suspended и не попадает в runnable queue до события. Поэтому файберный runtime должен использовать non-blocking I/O, epoll/io_uring и fiber-aware синхронизацию: блокироваться должен файбер, а не поток ОС.

Fault injection — это способ тестировать concurrent-код, искусственно вставляя `yield` в разные места или управляя scheduler-ом. Так можно детерминированно проверять разные interleavings, ловить race condition, deadlock, livelock и воспроизводить найденные ошибки по seed.

31. Мини-проверка

1. Что такое фибер?
2. Чем фибер отличается от потока ОС?
3. Почему фиберы обычно cooperative, а не preemptive?
4. Что нужно сохранить при context switch?
5. Почему достаточно сохранять `RSP` и callee-saved registers, а `RIP` явно не сохраняют?
6. Зачем фиберу свой стек?
7. Что делает `SetupContext` ?
8. Зачем нужен trampoline?
9. Какие базовые состояния есть у фибера?
10. Что делает scheduler?
11. Как работает `yield` ?
12. Чем `yield` отличается от `suspend` ?
13. Почему нельзя вызывать blocking `read` внутри фибера?
14. Как фиберный runtime интегрируется с `epoll` ?
15. Что такое fiber-aware mutex?
16. Что такое fault injection и зачем он нужен?
17. Почему fault injection удобно делать именно с фиберами?
18. Чем fault injection отличается от ThreadSanitizer?